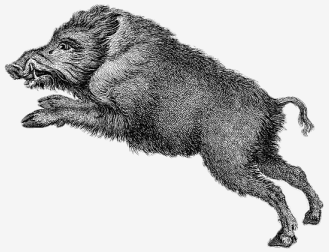
vk tech talks

Место встречи профессионалов

| Java

Кабанчиком по CRDT:

что такое Local First, или зачем мерджить JSON?



Георгий Семенов

Разработчик в RecSys Infra @ VK AI

- Студент ИПКН ИТМО и ШАД
- Поступаю в аспирантуру по CRDT
- Преподавал проектное программирование школьникам
- Люблю велосипед и бегать

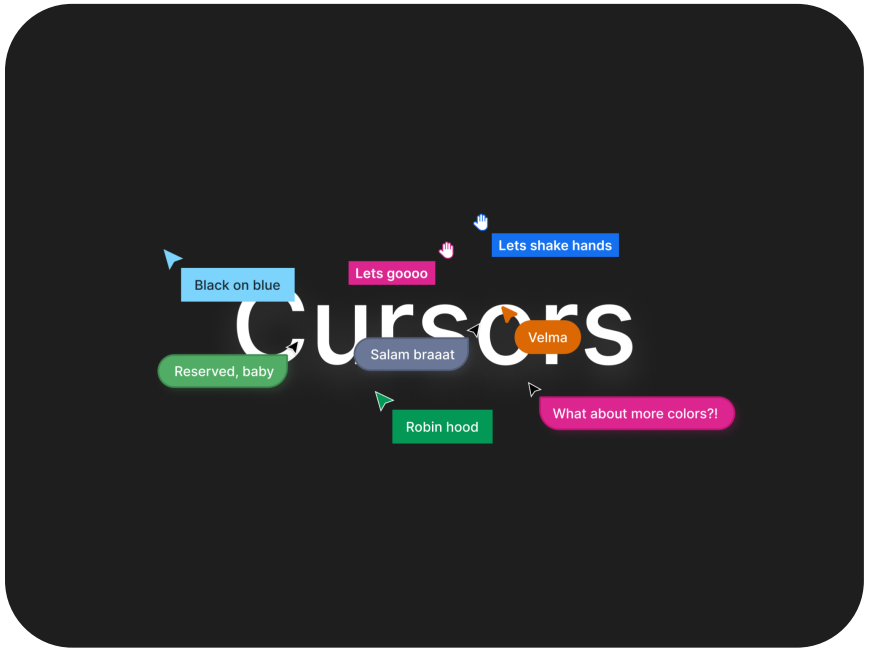
Roadmap

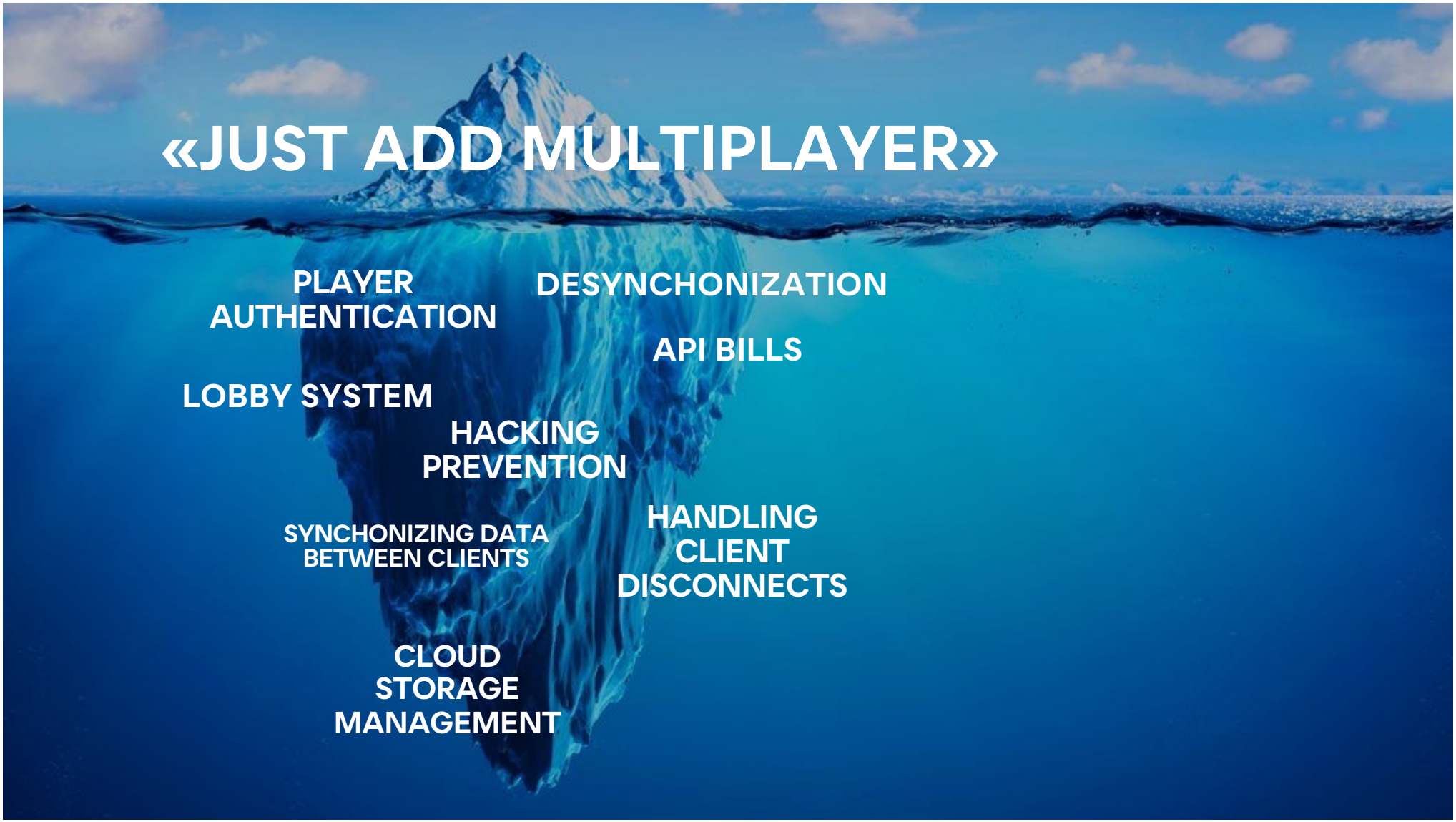
- Коллаборация в приложениях
- Синхронизация данных
- CRDT
- Local-First
- Итоги

Коллаборация в приложениях

Коллаборативные приложения

- Актуальное состояние с синхронизацией в реальном времени
- Мгновенная доступность данных даже с прерванным подключением
- Интерактивное взаимодействие между пользователями




«JUST ADD MULTIPLAYER»
PLAYER AUTHENTICATION
DESYNCHONIZATION
API BILLS
LOBBY SYSTEM
HACKING PREVENTION
SYNCHONIZING DATA BETWEEN CLIENTS
HANDLING CLIENT DISCONNECTS
CLOUD STORAGE MANAGEMENT

CAP-теорема

CAP-теорема [Эрик Брюер, 2000]

В системе возможно обеспечить не более двухсвойств из списка:

A – Availability

Система обладает свойством доступности, т.е. гарантирует наличие ответа за обозначенное время.

Система без этого свойства может «виснуть» (Timeout) или «лежать» (Denial of Service).

C – Consistency

Система обеспечивает согласованность данных во всех ее вычислительных узлах.

От системы без этого свойства в один момент времени можно получить противоречащие ответы.

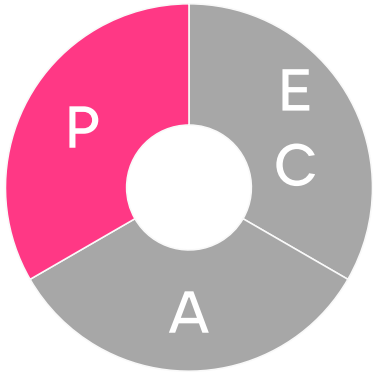
P – Partition Tolerance

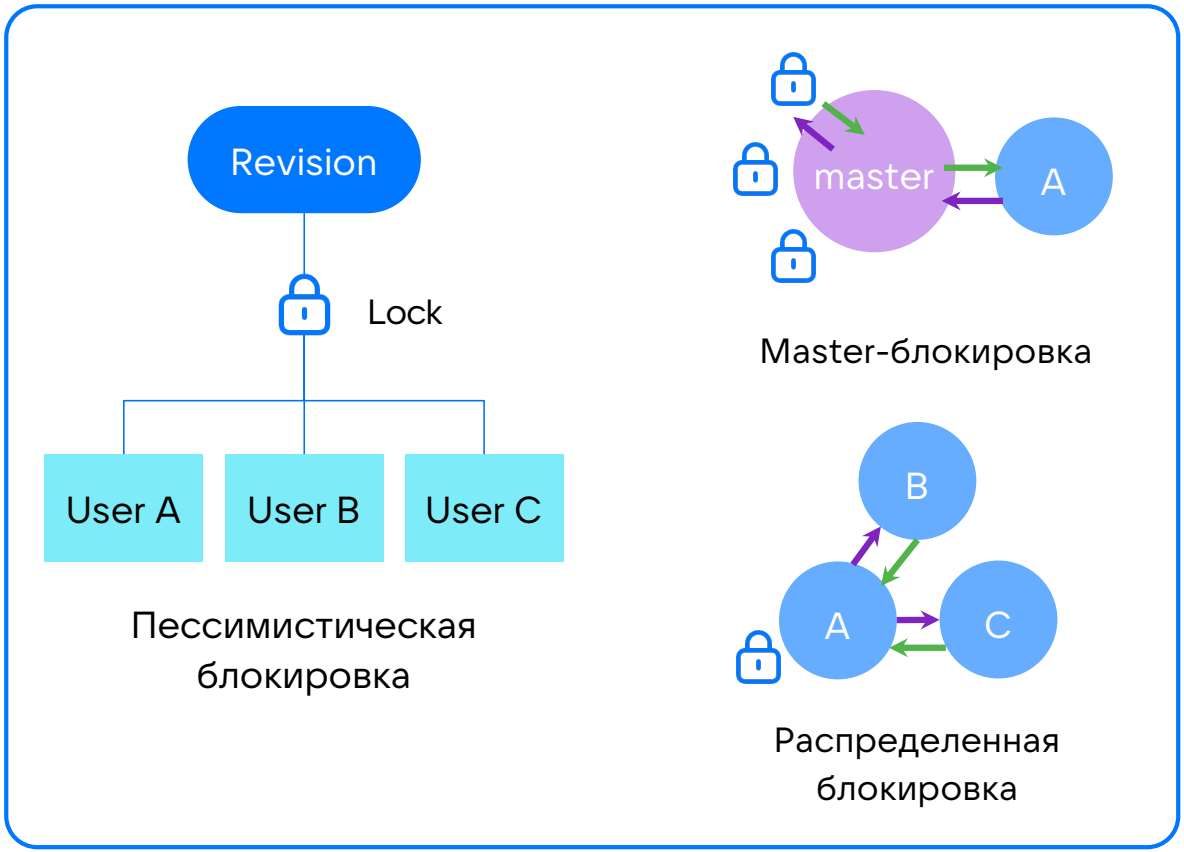
Система продолжает функционировать при частичном отказе ее вычислительных узлов.

Без этого свойства система перестанет функционировать, например, если отвалится диск, память или сеть.

Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services

Пессимистическая репликация

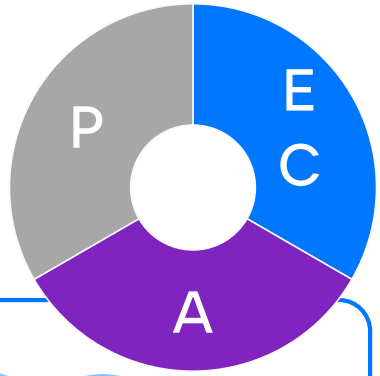




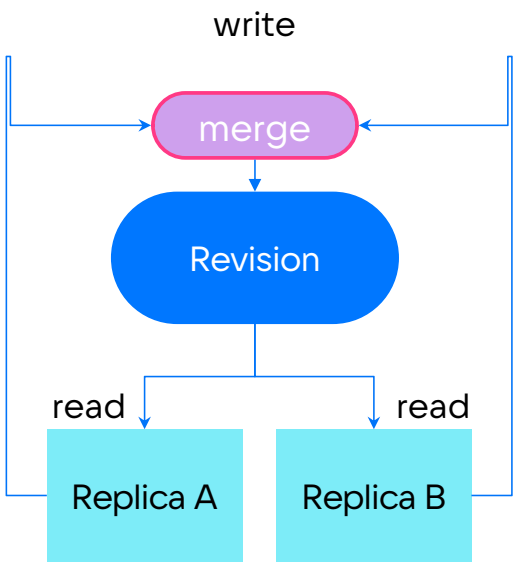
Где можно встретить?

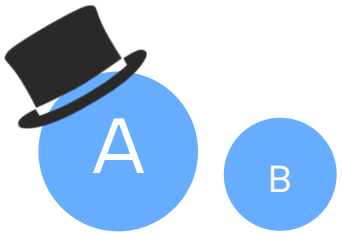
- СУБД
- VCS
- CAE-системы

Оптимистическая репликация

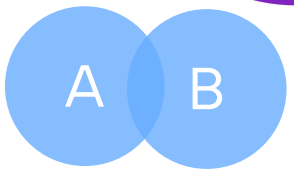


Разрешим локальную работу над данными в оптимистическом предположении о том, что мы сможем объединить получившиеся изменения.

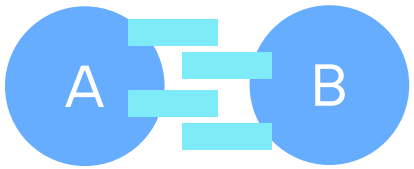




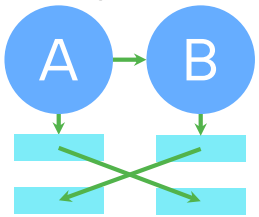
Приоритеты



Разрешение конфликтов



Тотальное упорядочивание



Каузальное упорядочивание

Saito, Y., & Shapiro, M. (2005). Optimistic replication

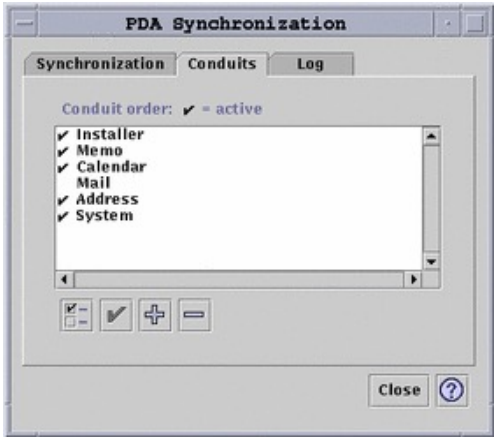
Синхронизация данных

«There is a lost chapter of computer science – data synchronization»

Victor Grishchenko

Зарождение научной синхронизации

Органайзер Oregon PDA-293 (2001)

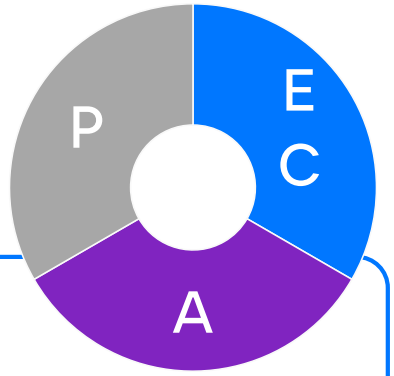


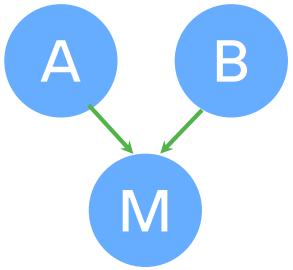
PDA Sync в составе Oracle Solaris (2010)

Starobinski, D., Trachtenberg, A., & Agarwal, S. (2003). Efficient PDA synchronization. IEEE Transactions on Mobile Computing, 2(1), 40-51.

Разрешение конфликтов оффлайн I

Консолидировать конкурентные изменения иногда очень просто.





2-Way Merge

```
Set.of(
  1, 2
)
```

+

```
Set.of(
  2, 3
)
```

=

```
Set.of(
  1, 2, 3
)
```

```
{
  "a": 1
}
```

+

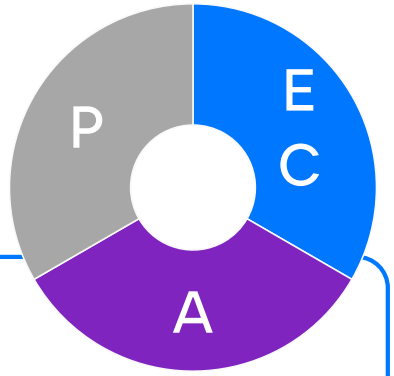
```
{
  "b": 2
}
```

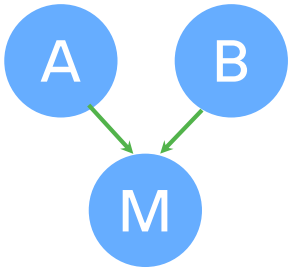
=

```
{
  "a": 1,
  "b": 2
}
```

Разрешение конфликтов оффлайн II

А иногда – результат в постановке 2-Way Merge трудно определить корректно.





2-Way Merge

```
{                 {
  "a": 1    +       "a": 2
}                 }
```

=

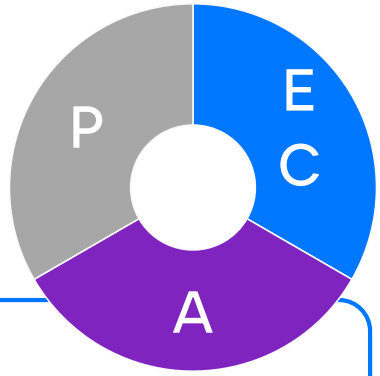
```
{           {
  "a": 1      "a": 2
}           }
```

```
"Hello, A!"  +  "Hello, B!"
```

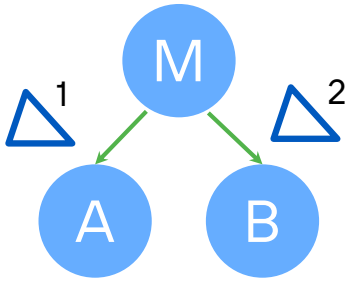
=

```
"Hello, A!"   "Hello, B!"
"Hello, AB!"   "Hello, BA!"
???
```

Разрешение конфликтов оффлайн III



Чуть лучше выглядит ситуация с 3-Way Merge – этот подход лежит в основе утилиты diff3, реализующей алгоритм Майерса. Это наш любимый git!



$$\Delta = \Delta^1 \cup \Delta^2 - \underbrace{\Delta^1 \cap \Delta^2}_{\text{conflict}}$$

3-Way Merge

```
[
  "Hi!",
  "Bye!"
]
```

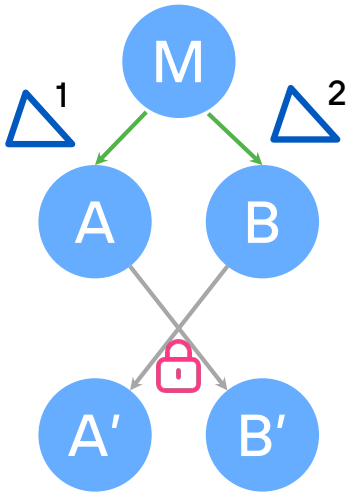
```
[
  "Hi!",
+  "I am A!",
  "Bye!"
]
```

```
[
  "Hi!",
+ "I am B!",
- "Bye!”,
+ "Bye, C!"
]
```

← Конкурентные вставки и удаления порождают конфликты.

Myers, E. W., & Miller, W. (1989). Approximate matching of regular expressions

Почему конфликты – это боль?



← В общем случае перед продолжением работы необходимо разрешить конфликт

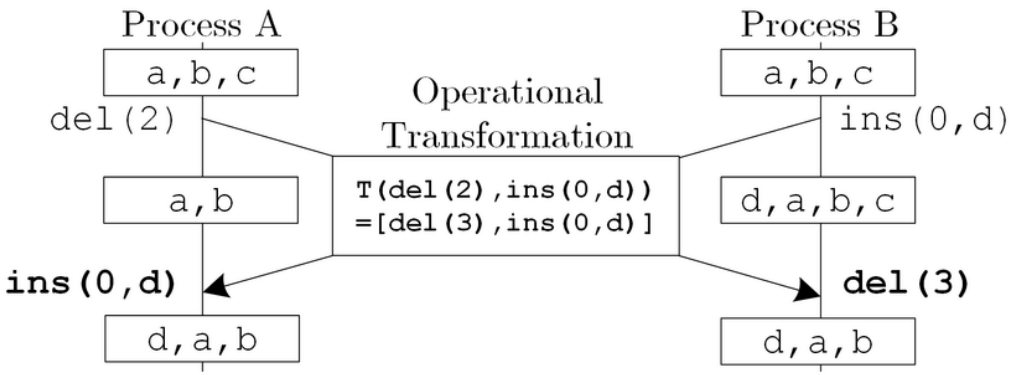
OT = Operational Transformations

Пусть у нас есть **онлайн-канал связи**, через который локальные операции распространяются на все реплики.

Определим **преобразование на операциях**, которая позволяет переставлять их местам.

Декларируемые свойства OT:

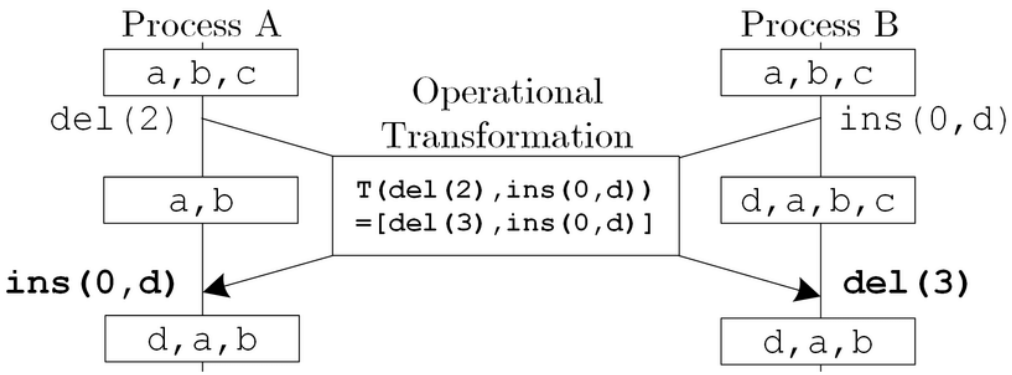
- Convergence
- Intention preservation
- Causality preservation



Kumawat, S., & Khunteta, A. (2010). A survey on operational transformation algorithms: Challenges, issues and achievements. International Journal of Computer Applications, 3(12), 30-38.

OT = Operational Transformations

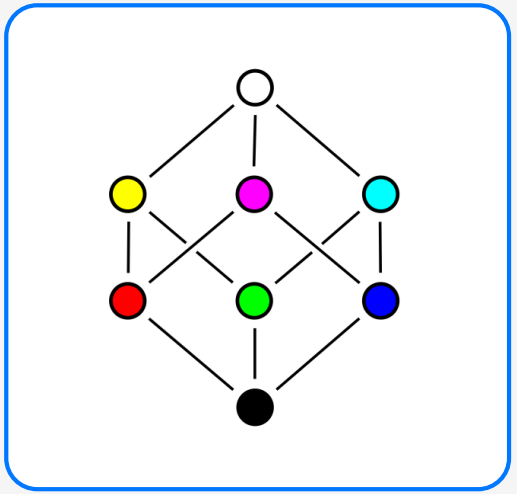
- Нужен центральный сервер
- Необходимо хранить полную историю версий
- Очень сложно
- Некорректные реализации, потому что слишком сложно доказывать в общем виде



- До сих пор в Google Docs 😏

CRDT

= conflict-free replicated data types



Мотивация к CRDT

Что мы хотим от реплицируемых типов?

- Никаких трансформаций над операциями – это error-prone approach
- Не хотим обязательной централизации
- Пусть всегда и естественно сходится – **strong eventual consistency**
- Возможность формально доказать соблюдение необходимого свойства

Навстречу CRDT

2011

Марк Шапиро публикует статью «Conflict-free Replicated Data Types»

1979
diff3
Утилита для 3-Way Merge в составе Unix (Version 7)

1996
rsync
Утилита синхронизации данных в составе linux

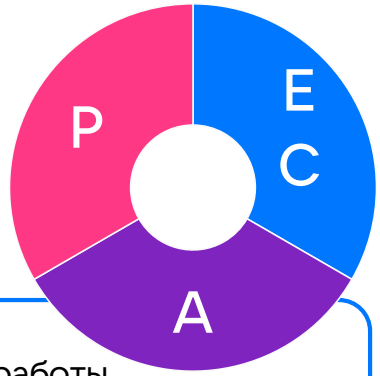
2003
WebDAV + CalDAV
Создаются открытые протоколы для синхронизированной работы с файловыми хранилищами и календарями

2006
Google Docs
Онлайн-коллаборация на основе OT

2007
Amazon Shopping Cart
Выходит статья «Amazon's Highly Available Key-value Store»

2009
Apache Wave
Конец эпохи OT = Operational Transformations

CRDT



CRDT (conflict-free replicated datatypes) – структуры данных для коллаборативной работы, обладающие свойством сильной конечной согласованности.

Что значит «conflict-free» и «strong eventual consistency» (SEC)?

- Реплики сойдутся к одному единому состоянию в присутствии сетевого прогресса
- Без консенсусов, кворумов и прямой синхронизации
- Никто не гарантирует соблюдения первоначальных пользовательских намерений

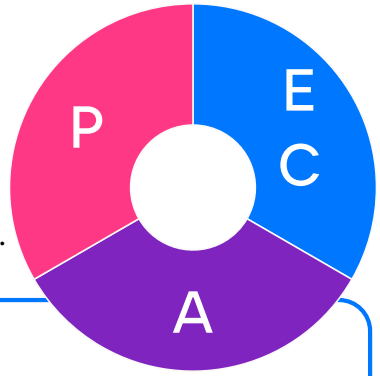
Любой экземпляр CRDT обладает:

- Identity – глобально уникальный идентификатор
- Payload – функционально чистое сериализуемое состояние

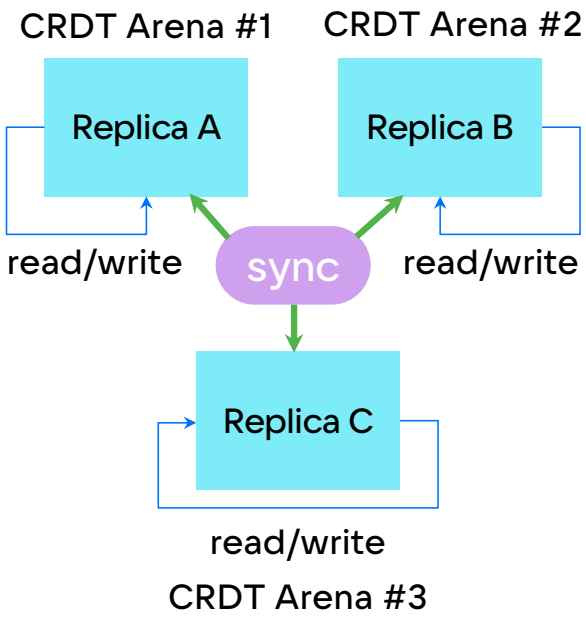
```
trait CRDT[S] {
   def id() : RIdentity
   def apply(): S
}
```

Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). A comprehensive study of convergent and commutative replicated data types

Коллаборация на основе CRDT



Давайте разрешим работать над данными подлинно параллельно, определив «хороший» метод синхронизации данных – слияние CvRDT / бродкаст операций CmRDT.



По запросу создается централизованный коллаборативный рантайм на основе CRDT – в фоне лог операций компактифицируется и стабилизируется

Например, CRDT в централизованном контексте используют в Figma

Зоопарк CRDT

- **Counters:**
- Grow-Only (GCounter)
- Positive-Negative (PNCounter)

- **Registers:**
- Last-Writer-Wins (LWWRegister)
- Multi-Value (MVRegister)

- **Sets:**
- Grow-Only (GSet)
- Two-Phase (2PSet)
- Unique-Element (USet)
- Last-Writer-Wins (LWWSet)
- Positive-Negative (PNSet)
- Observed-Remove (ORSet)

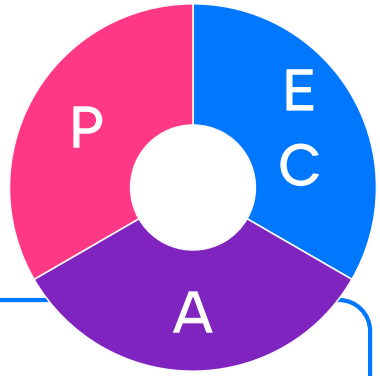
- **Miscellaneous:**
- Add-Remove Partial Order
- Causal Trees

- **Graphs:**
- 2P-Set Graph (2P2P-Graph)
- Add-only monotonic DAG

- **Lists / Text collaborative editing:**
- Continuous sequence
- WOOT[1] = Without OT
- RGA[2] = Replicated Growable Array
- Logoot[3]
- Treedoc[4]
- Chronofold[5]
- Fugue/FugueMax[6] etc.

[1] Oster, G., Urso, P., Molli, P., & Imine, A. (2006) Data consistency for P2P collaborative editing.
[2] Roh, H. G., Kim, J. S., Lee, J., & Maeng, S. (2009). Optimistic operations for replicated abstract data types.
[3] Weiss, S., Urso, P., & Molli, P. (2009) Logoot: A scalable optimistic replication algorithm for collaborative editing on p2p networks.
[4] Preguiça, N., Marquès, J. M., Shapiro, M., & Letia, M. (2009). A commutative replicated data type for cooperative editing.
[5] Grishchenko, V., & Patrakeev, M. (2020, April). Chronofold: a data structure for versioned text.
[6] Weidner, M., & Kleppmann, M. (2023). The art of the fugue: Minimizing interleaving in collaborative text editing.

CvRDT – state-based CRDTs



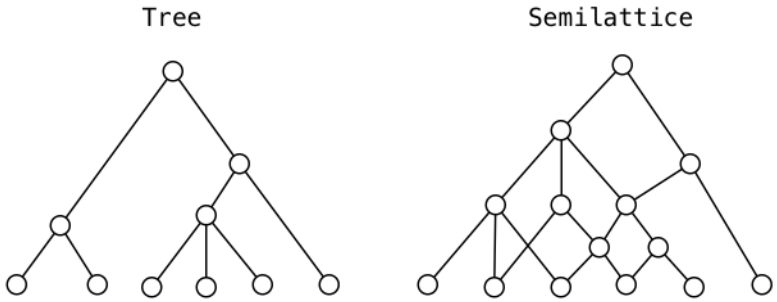
CvRDT (convergent replicated datatypes) – типы данных, над синхронизацией состояний которых и не нужно сильно заморачиваться.

```
trait CvRDT[T <: CvRDT[T, S], S] extends CRDT[S] {
   def merge(other: T): T
}
```

Состояния CvRDT образуют полурешетку (semilattice) – полугруппа с идемпотентной и коммутативной операцией.

У CvRDT из коробки есть операция merge =

- LUB = least upper bound
- GLB = greatest lower bound



Max – пример CvRDT

Max – регистр, принимающий максимальное из значений своих реплик:

```
case class Max[T : Ordering] (
  value: T,
  objectId: RIdentity = RIdentity.empty
)(
  implicit ordering: Ordering[T]
) extends CvRDT[Max[T], T] {

  override def id(): RIdentity = objectId

  override def merge(other: Max[T]): Max[T] = Max(
    ordering.max(value, other.value),
    objectId
  )(ordering)

  override def apply(): T = objectId
}
```

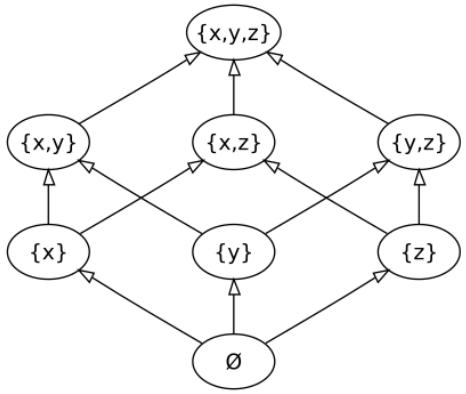
Grow-only Set – пример CvRDT

Grow-only Set – множество, в которое можно только добавлять новые элементы:

```
case class GSet[T](items: Set[T], objectId: RIdentity) extends CvRDT[GSet[T], Set[T]] {
  override def id(): RIdentity = objectId
  override def merge(other: GSet[T]): GSet[T] = GSet(items.union(other.items), objectId)
  override def apply(): Set[T] = items
}
```

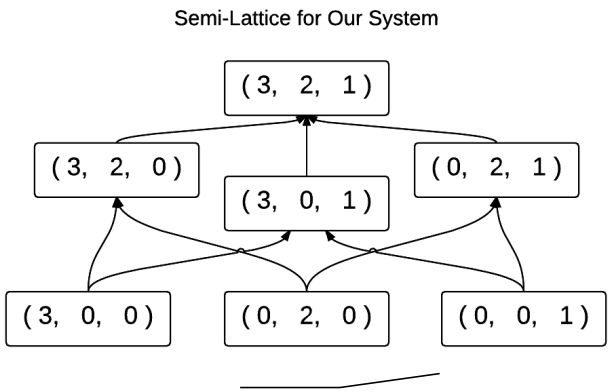
Как доказать свойство «strong eventual consistency»?

- Надо показать, что при сетевом прогрессе все реплики придут к одинаковому состоянию
- С CvRDT все просто – ссылаемся на математику и полурешетки



Grow-only Counter – пример CvRDT

```
case class GCounter[T](
  shards: Map[ReplicaId, Max[T]] = Map.empty,
  objectId: RIdentity = RIdentity.empty
)(
  implicit monoid: Monoid[T]
) extends CvRDT[GCounter[T], T] {

  override def id(): RIdentity = objectId

  override def apply(): T = shards.values
      .foldLeft(monoid.empty)
      ((r, shard) => monoid.combine(r, shard()))
```

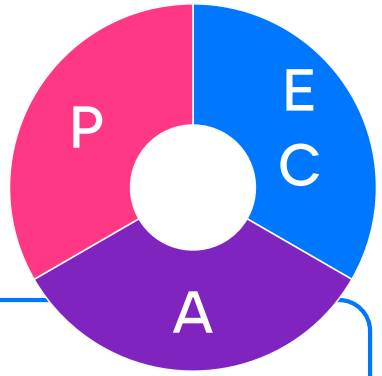


Значения счетчика по акторам — в свой шард может писать только соответствующая реплика

Grow-only Counter – пример CvRDT

```
override def merge(other: GCounter[T]): GCounter[T] = {
  val replicaIds = shards.keySet.union(other.shards.keySet)

  val mergedShards: Map[ReplicaId, Max[T]] = replicaIds.map { replicaId =>
    val left = shards.get(replicaId)
    val right = other.shards.get(replicaId)

    val merged = (left, right) match {
      case (Some(l), Some(r)) => l.merge(r)
      case (Some(l), None) => l
      case (None, Some(r)) => r
      case (None, None) => throw new IllegalStateException()
    }

    replicaId -> merged
  }.toMap

  GCounter(mergedShards, objectId)
}
```

CmRDT – op-based CRDTs



CmRDT (commutative replicated datatypes) – типы данных, состояние которых описывается последовательностью «коммутативных» операций.

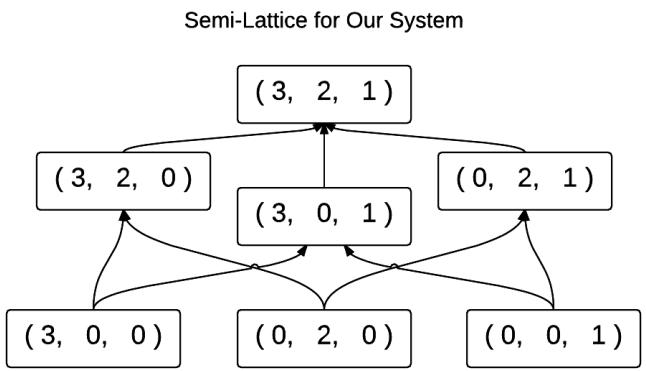
```
trait CmRDT[T <: CmRDT[T, Op, S], Op, S] extends CRDT[S] {
   def accept(op: Op): T
}
```

Коммутативность операций A и B по отношению к состоянию S:

$$S \circ A \circ B = S \circ B \circ A$$

Grow-only Counter – пример CmRDT

```scala
case class GCounter[T](
  shards: Map[ReplicaId, T] = Map.empty,
  objectId: RIdentity = RIdentity.empty
)(
  implicit monoid : Monoid[T]
) extends CmRDT[GCounter[T], GCounter.Op[T], T] {

  override def id(): RIdentity = objectId

  override def apply(): T = shards.values
    .foldLeft(monoid.empty)
    ((r, shard) => monoid.combine(r, shard))
```



Grow-only Counter – пример CmRDT

```
  override def accept(op: GCounter.Op[T]): GCounter[T] = op match {
    case Increment(add : T) => GCounter {
      shards.transform((replicaId, shard) => {
        if (replicaId == op.replica()) monoid.combine(shard, add)
        else shard
      })
    }
  }
}

object GCounter {
  sealed trait Op[T] extends OpMetadata

  case class Increment[T](add: T)(implicit val ctx : RContext) extends Op[T] with WithIncrement[T] {
    override def replica(): ReplicaId = ctx.me()
    override def incCount(): T = add
  }
}
```

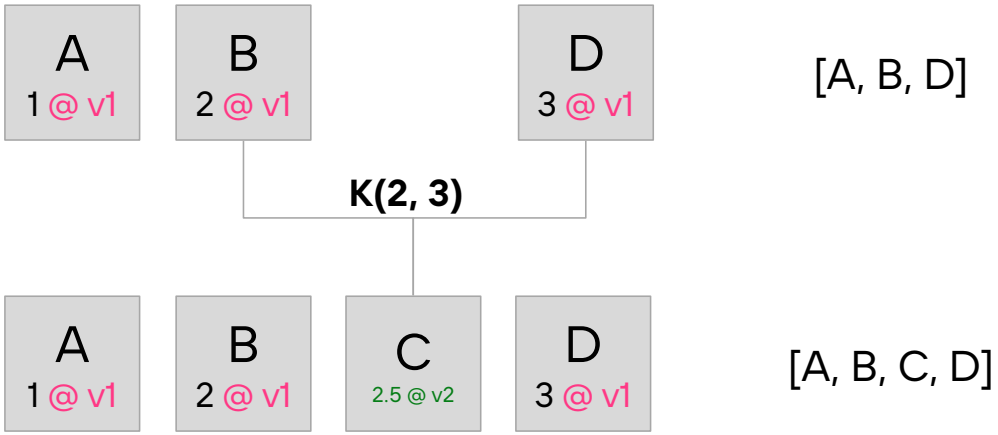
Регистр Last-Write-Wins

```
case class LWWRegister[T](
   value: T,
   timestamp: Instant = Instant.now(),
   objectId: RIdentity = RIdentity.empty
 ) extends CvRDT[LWWRegister[T], T] {

  override def id(): RIdentity = objectId

  def write(newValue: T, newTimestamp: Instant = Instant.now()): LWWRegister[T] = {
    if (newTimestamp.isAfter(timestamp)) {
      LWWRegister(newValue, newTimestamp)
    } else {
      this
    }
  }

  override def merge(other: LWWRegister[T]): LWWRegister[T] = {
    if (other.timestamp.isAfter(this.timestamp)) other else this
  }
}
```

Реплицированные списки

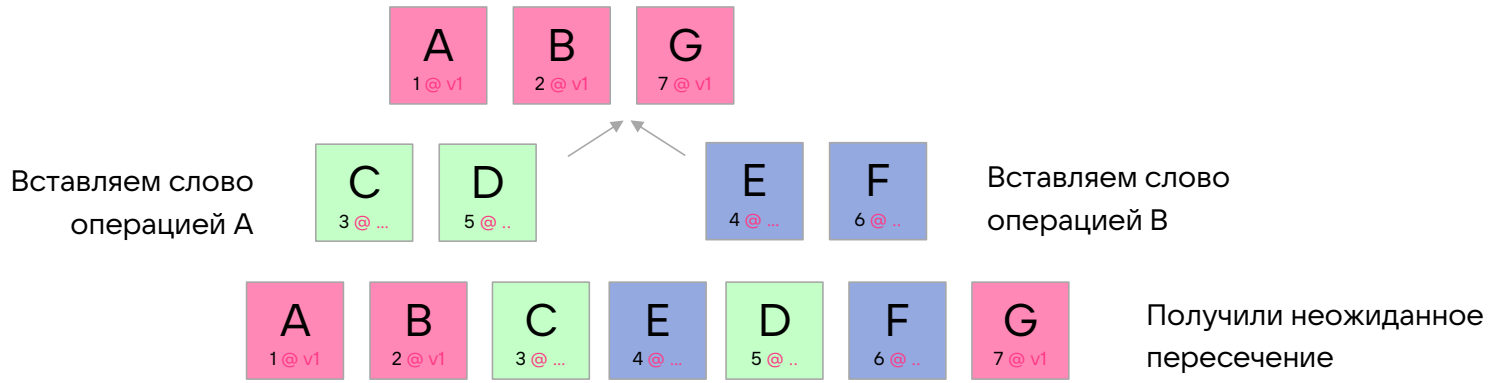
Основная идея для списка – использование плотного множества тэгов для упорядочения элементов.

```
+ LWWRegister
+ GSet
+ GCounter
```



Проблема «перекрытий» (interleavings)

Рассмотрим пример с конкурентными вставками элементов в список на основе continuous sequence.



Local-First

LOCAL-FIRST APPS
DEVELOPERS
SYNC ENGINE COMPLEXITY
DEVELOPERS
LOCAL-FIRST APPS

Навстречу Local-First

2019

Мартин Клеппман публикует статью «Local-First Software: You Own Your Data, in spite of the Cloud»

2010
Dropbox и др.
Идея синхронизации файлов находит признание

2012
DynamoDB
NoSQL – и точка

2015
Progressive Web Applications
Зарождение подхода к использованию мощностей браузеров для создания приложений

2016
A Conflict-Free Replicated JSON
Статья М. Клеппмана

2021
Yjs & AutoMerge
Фреймворки для построения коллаборативных приложений на основе CRDT

2024
Local First Conf
Первая тематическая конференция

Манифест Local-First

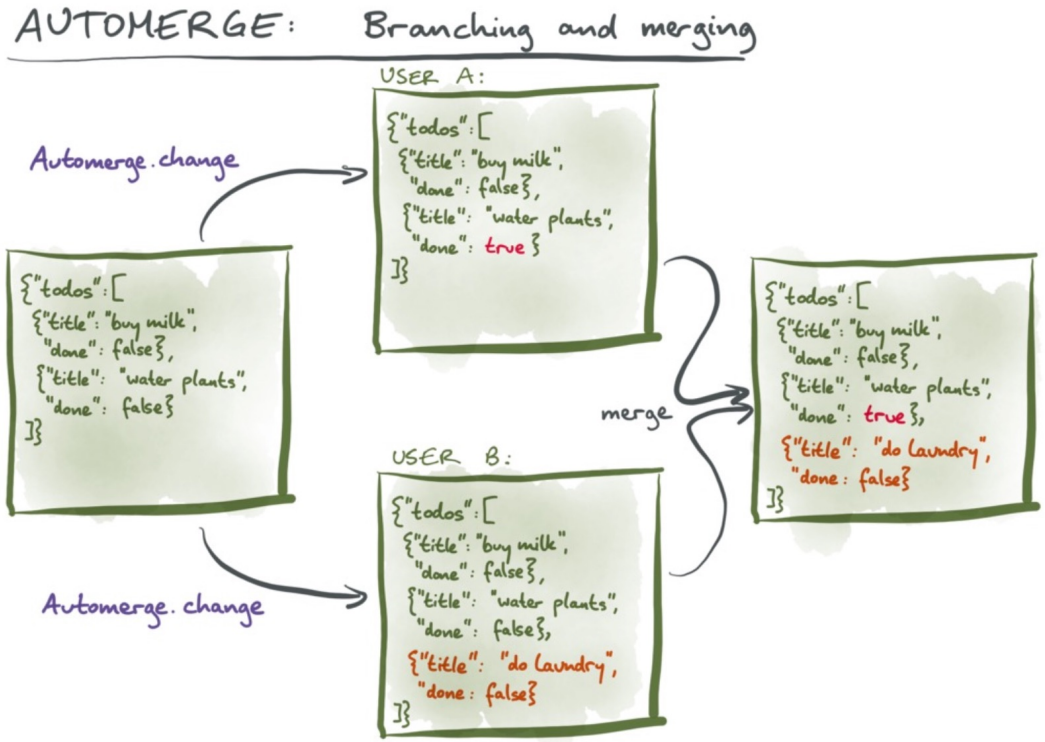
7 идеалов Local-First программного обеспечения:

1. Нет «спиннерам» – все данные уже на месте
   (хватит гонять трафик по ДЦ)
2. Твоя работа не заложник одного устройства
   (хранить данные без возможности синхронизации – глупость)
3. Сеть опциональна
   (само досинхронизируется, когда сеть вернется)
4. Бесшовная коллаборация с коллегами
   (синхронизация правок в реальном времени – CRDT)
5. «Непрерывная вечность» данных
   (просмотреть данные можно вечно – JSON, XML и т.п.)
6. Безопасность и конфиденциальность по умолчанию
   (возможность выборочного доступа благодаря peer-to-peer)
7. Вы полностью владеете своими данными
   (бэкапы делать вам)

Мартин Клеппман и его книга «Высоконагруженные приложения»
(aka «кабанчик»)

M. Kleppmann et al. (2019) Local-first software: you own your data, in spite of the cloud

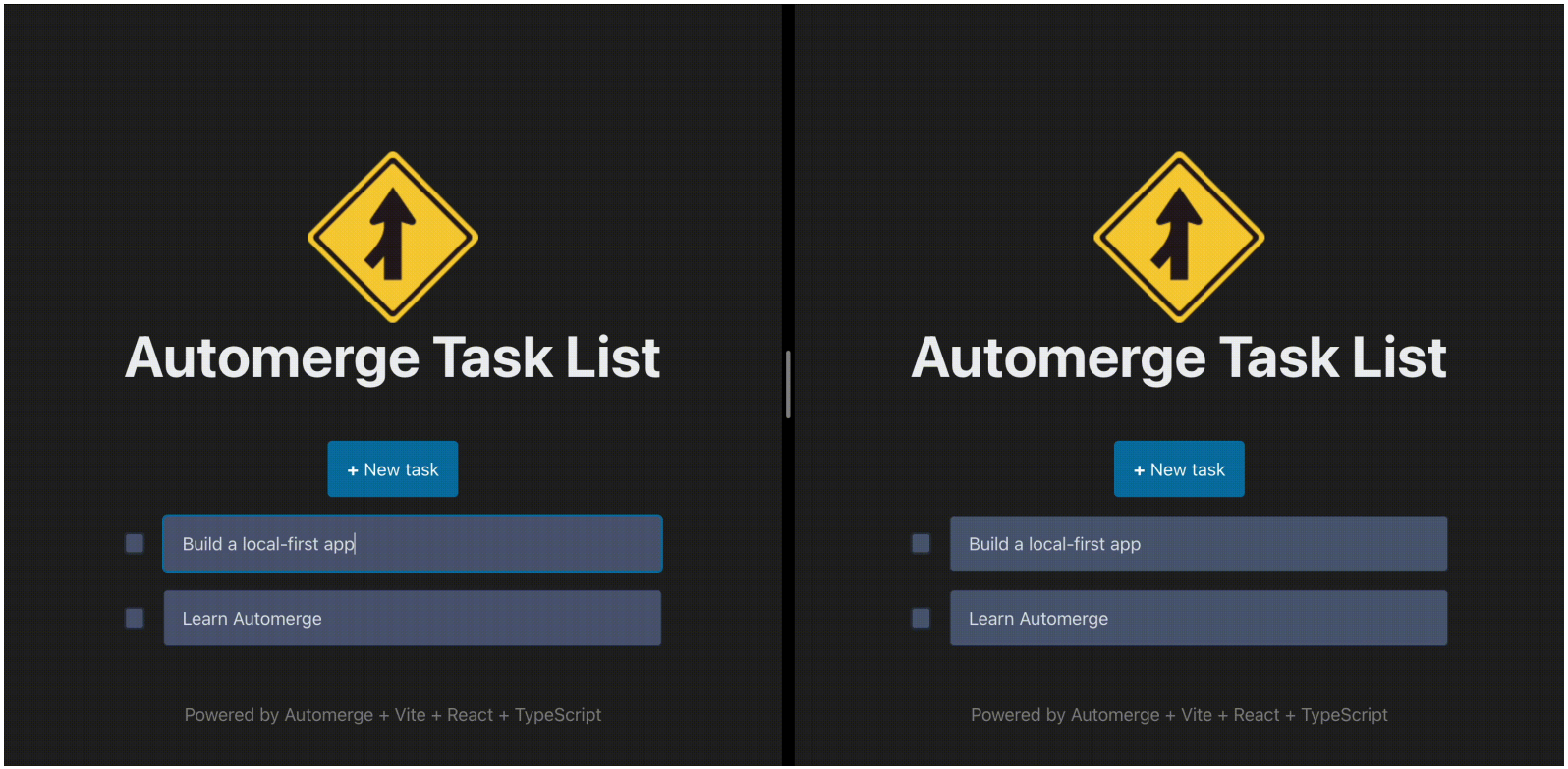
Время мерджить JSON



JSON как композиция реплицируемого списка и регистра

Источник: Martin Kleppmann. Automerge: a new foundation for collaboration software

AutoMerge



https://github.com/automerge/automerge-java

AutoMerge

```java
// Create an object
Document doc = new Document();

ObjectId text;
try(Transaction tx = doc.startTransaction()) {
    // Create a text object under the "text" key of the root map
    text = tx.set(ObjectId.ROOT, "text", ObjectType.TEXT);
    tx.spliceText(text, 0, 0, "Hello world");
    tx.commit();
}

// save the document
byte[] docBytes = doc.save();
```

https://github.com/automerge/automerge-java

AutoMerge

```
// Load the document
Document doc2 = Document.load(docBytes);
System.out.println(doc2.text(text).get()); // Prints "Hello world"

// Modify the doc in doc2
try(Transaction tx = doc2.startTransaction()) {
    tx.spliceText(text, 5, 0, " beautiful");
    tx.commit();
}

// Modify the doc in doc1
try(Transaction tx = doc.startTransaction()) {
    tx.spliceText(text, 5, 0, " there");
    tx.commit();
}

// Merge the changes
doc.merge(doc2);

// Prints either "Hello there beautiful world" or "hello beautiful there world"
// depending on the actor IDs that were generated for each document.
System.out.println(doc.text(text).get());
```

Как работает AutoMerge

AUTOMERGE: "Git for your app's data"

```
{"todos": [
    {"title": "buy milk",  "done": false},
    {"title": "water plants",  "done": false},
    {"title": "do laundry",  "done": false}
]}
```

added

```
after = Automerge.change(before, "add new item", doc => {
    doc.todos.push({title: "do laundry", done: false});
});
```

Источник: Martin Kleppmann. Automerge: a new foundation for collaboration software

Как работает AutoMerge

```
Automerge.change(state, doc => {
    doc.todos.push({title: "do laundry", done: false});
});
```

operation log

```
{op: "makeMap", id: "5a", obj: "1a", elemID: "2a", insert: true}
{op: "assign", id: "6a", obj: "5a", key: "title", value: "do laundry"}
{op: "assign", id: "7a", obj: "5a", key: "done", value: false}
```

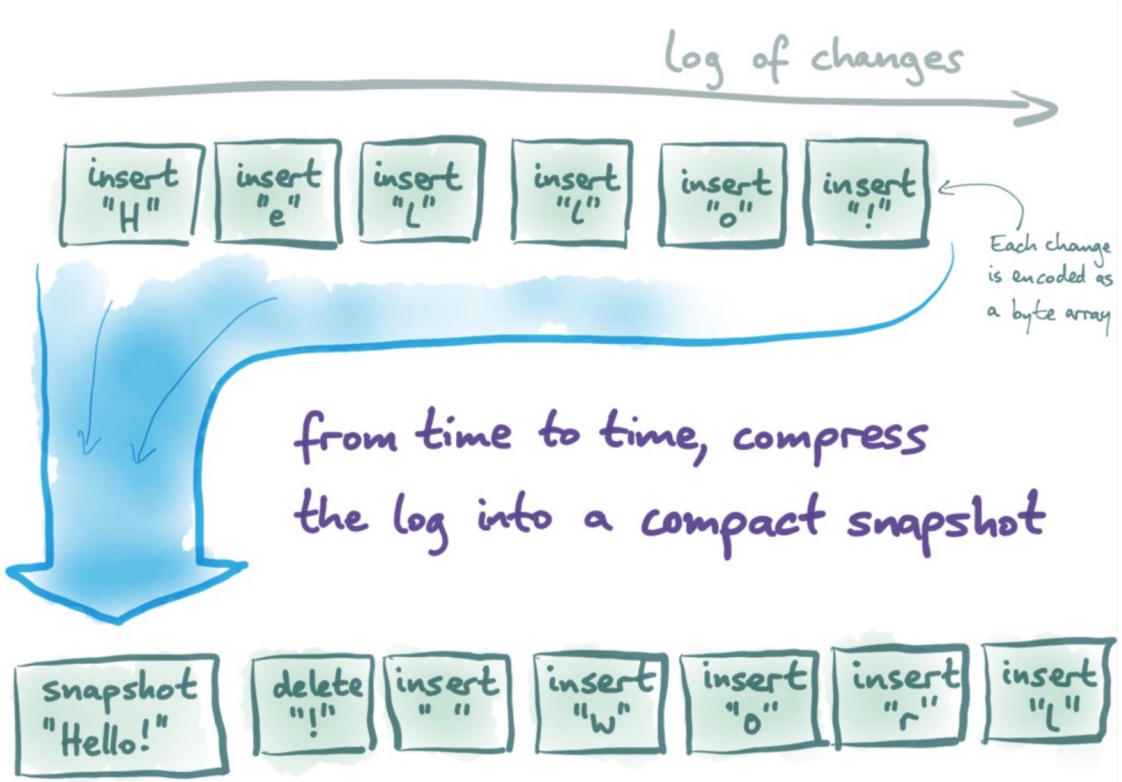
compressed binary encoding

```
Uint8Array([0x85, 0x6f, 0x4a, 0x83, ...])
```

write to disk, send over network

Источник: Martin Kleppmann. Automerge: a new foundation for collaboration software

Как работает AutoMerge



Источник: Martin Kleppmann. Automerge: a new foundation for collaboration software

Y.js

Y.js – фреймворк для построения CRDT-приложений в вебе из коробки.

index.ts × y-quill.stackblitz.io

```
9   const quill = new Quill(document.querySelector
    ('#editor'), {
10    modules: {
11      cursors: true,
12      toolbar: [
13        // adding some basic Quill content features
14        [{ header: [1, 2, false] }],
15        ['bold', 'italic', 'underline'],
16        ['image', 'code-block']
17      ],
18      history: {
19        // Local undo shouldn't undo changes
20        // from remote users
21        userOnly: true
22      }
23    },
24    placeholder: 'Start collaborating...',
25    theme: 'snow' // 'bubble' is also great
26  })
```

Normal B I U

Start collaborating...

Console

Edit on StackBlitz Editor Preview Both

index.ts × y-quill.stackblitz.io

```
28  // A Yjs document holds the shared data
29  const ydoc = new Y.Doc()
30
31  const provider = new WebrtcProvider
    ('quill-demo-room', ydoc)
32
33  // Define a shared text type on the document
34  const ytext = ydoc.getText('quill')
35
36  // "Bind" the quill editor to a Yjs text type.
37  const binding = new QuillBinding(ytext, quill,
    provider.awareness)
38
39  // Remove the selection when the iframe is blurred
40  window.addEventListener('blur', () => { quill.blur
    () })
41
```

Normal B I U

Start collaborating...

Console

Edit on StackBlitz Editor Preview Both

Итоги

Итоги

- Коллаборативные приложения любят Local First
- CRDT – математический аппарат для размышления о корректных реплицируемых типах
- Можно поиграться с коллаборативным Runtime с помощью Automerge / Y.js